

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

February 24, 2026
Tuesday



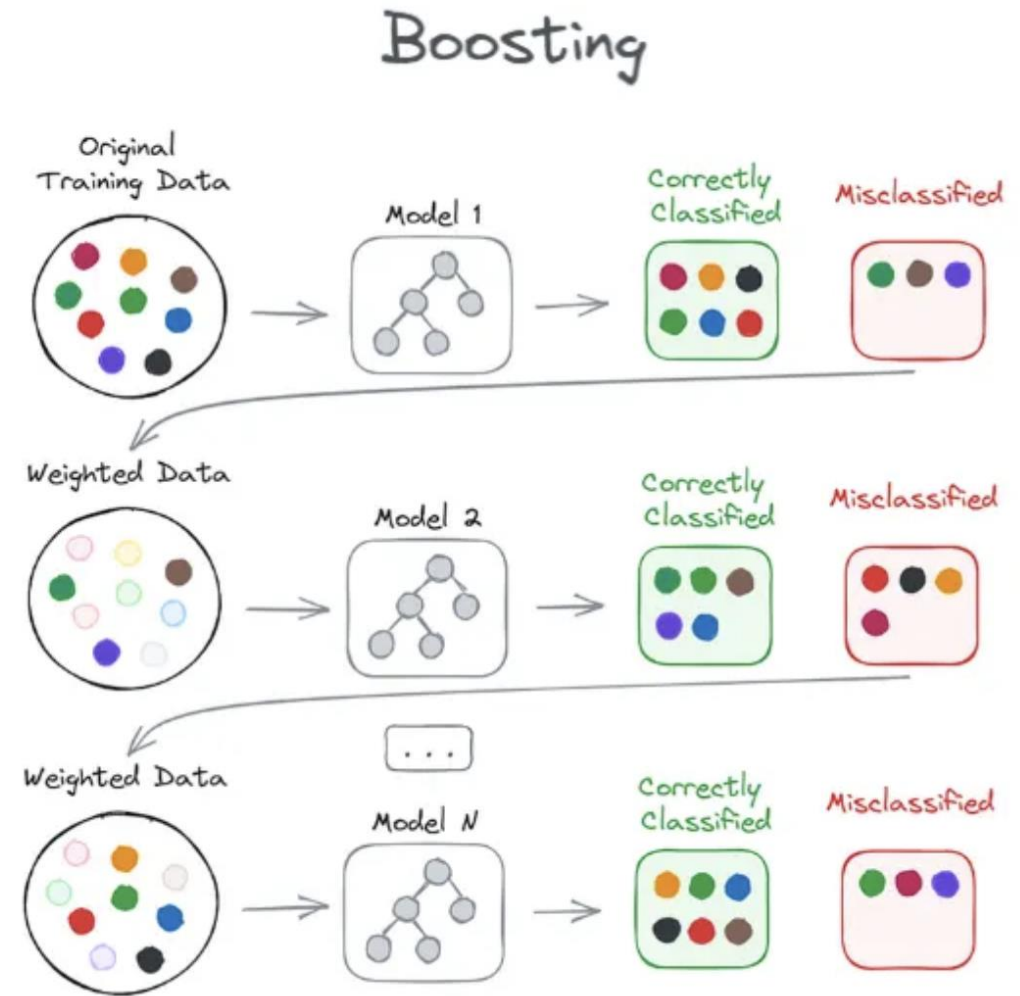
Today's goal – Learn about gradient boosted classification and kernels

- (1) Revisiting Gradient Boosted Regression
- (2) Gradient Boosted Classification
- (3) Class Activity: Gradient boosted trees
- (4) Introduction to Kernels
- (5) Kernelized Perceptron

Gradient boosted trees

Boosting

Boosting in machine learning is an ensemble technique that **sequentially builds a strong predictive model** by combining multiple "weak learners" (usually shallow decision trees), where each **new model attempts to correct the errors of its predecessor**.



Gradient Boosting

Gradient Boosting builds models sequentially, where each new model fixes the mistakes of the previous ones.

Gradient Boosting

1. Initialization:

- The ensemble starts with an initial model, often a simple one like a single-leaf tree or a constant value
- The initial prediction is made by this base model

Start with a simple guess.

2. Sequential Training:

For each iteration:

- 1.Compute the residuals:** Calculate the difference between the actual target values and the predictions made by the current ensemble.
- 2.Create a new tree (Fit a base learner):** Train a new learner (typically a decision tree) on the residuals. Fit the new model to predict the residuals, rather than the original target values.
- 3.Calculate the output values:** In this part, we determine the output values for each leaf.
- 4.Make a new prediction:** Update the ensemble by adding the new model's predictions scaled by a learning rate.
- 5.Repeat** until a predefined number of iterations (trees) is reached or until a stopping criterion is met.

What did we get wrong?

Build a tree that focuses on the error

Keep adding small correction models until predictions are good

3. Combining Models:

- The final prediction is the sum of the initial prediction and the predictions from all subsequent models

Final model is base guess + all the corrections

Gradient boosted regression

Gradient Boosted Regression

- Age prediction task

1. Predict **the average age** for everyone

(say average = 20 years) – not correct for all samples

2. Compute Errors (Residuals) = Actual value – Predicted value

Some ages:

- +10 (you underpredicted), -20 (you overpredicted)
- These residuals become the **new target**.

3. The next model learns the errors.

4. This shallow decision tree says:

Old samples → +10 correction, young samples → -20 correction

New Prediction = Old Prediction + Learning Rate × Tree Prediction

Repeat

Now compute new residuals again.

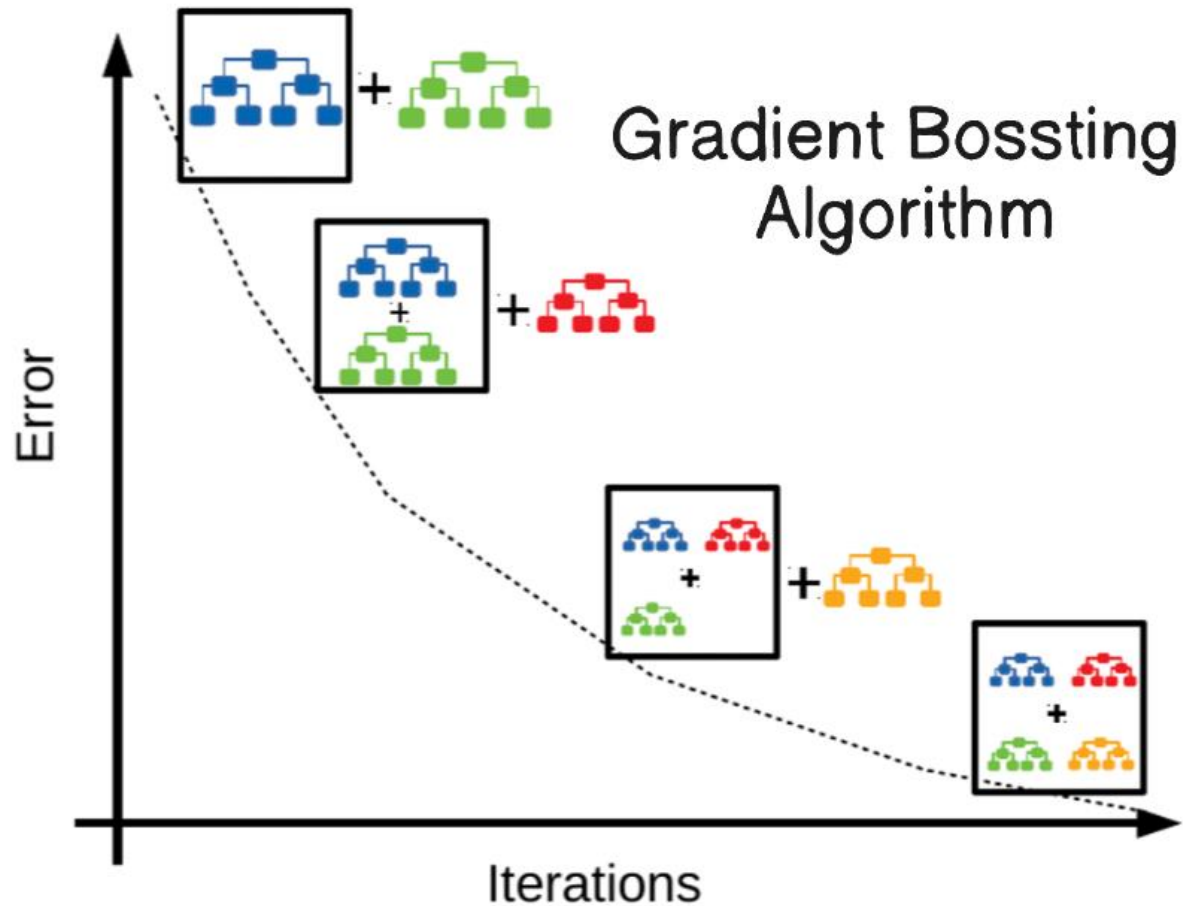
Train another tree.

Add it.

Repeat.

Gradients in Gradient Boosting

Gradient: The direction that reduces error the fastest.



Residuals are the gradient.

When we fit trees to residuals, we are following the gradient downhill to reduce error.

Gradients in Gradient Boosting

Squared loss for regression

$$L = \frac{1}{2}(y - \hat{y})^2$$

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = -(y - \hat{y})$$

$y - \hat{y} = \text{residual}$

negative gradient $= y - \hat{y}$

Cross entropy loss for classification

$$L = -y \log(p) - (1 - y) \log(1 - p)$$

Boosting is gradient descent in function space.

Instead of adjusting numerical parameters (like weights), we are adjusting an entire function.

Each boosting step adds a small function (a tree) in the direction that reduces loss fastest.

We want to reduce:

$$L(y, F(x))$$

So we compute the gradient with respect to the function:

$$\frac{\partial L}{\partial F(x_i)}$$

Any questions?



Gradient Boosted Classification

Gradient Boosted Classification

- Heart disease classification task

1. Predict **base probability** for everyone

(say 0.5) – not correct for all samples

2. Compute gradients (**Pseudo-Residuals**) = Actual label – Predicted probability

If true label 1: +0.5 (need to increase probability), 0: -0.5 (need to decrease probability),

These **pseudo-residuals** become the **new target**.

3. The next model learns the pseudo-residuals

4. This shallow decision tree says:

1 label samples → +0.5 correction, 0 label samples → -0.5 correction

New Prediction = Old Prediction + Learning Rate × **Tree Prediction**

Class activity

Gradient Boosting Demo

Introduction to Kernels

Linear functions may underfit the data



“CpG site 1”

$$x^{(1)} = 0.2$$

$$x^{(2)} = 0.3$$

$$x^{(3)} = 0.4$$

⋮

⋮

⋮

⋮

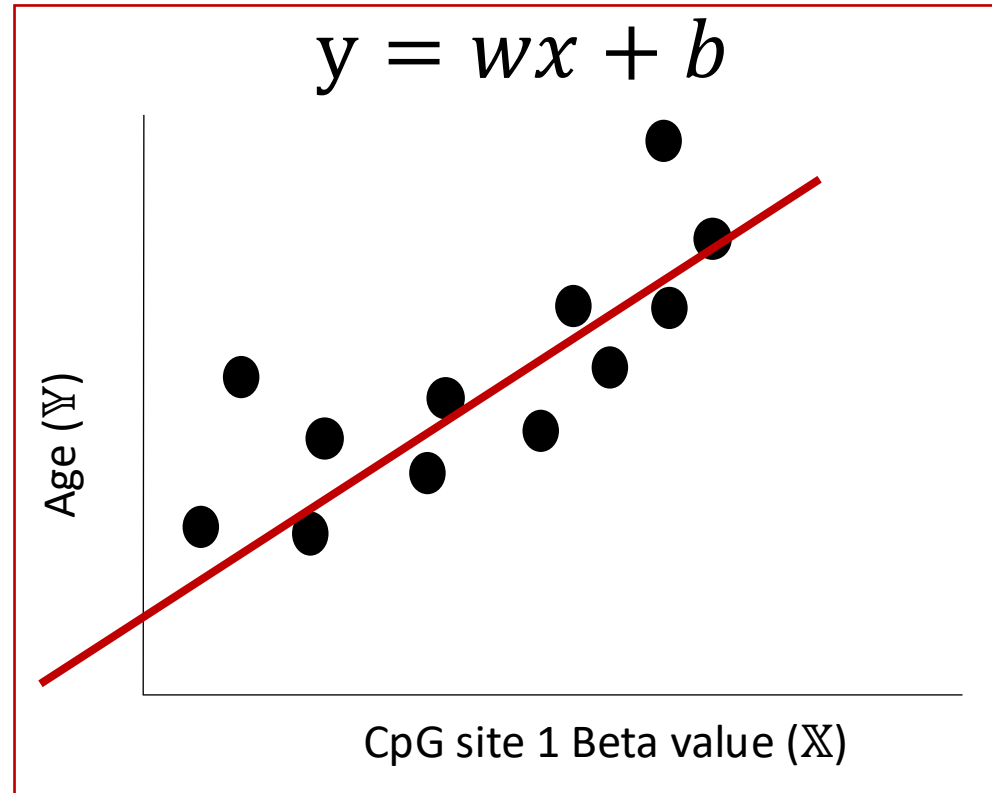
$$x^N = \dots$$

$$\mathbb{X} \in \mathbb{R}$$

What can we do here?

Linear function

$$y = wx + b$$



“Age”

$$y^{(1)} = 25$$

$$y^{(2)} = 35$$

$$y^{(3)} = 40$$

⋮

⋮

⋮

⋮

$$y^N = \dots$$

$$\mathbb{Y} \in \mathbb{R}$$

(Numerical output)

Learning better models – Try different functions



“CpG site 1”

$$x^{(1)} = 0.2$$

$$x^{(2)} = 0.3$$

$$x^{(3)} = 0.4$$

⋮

⋮

⋮

⋮

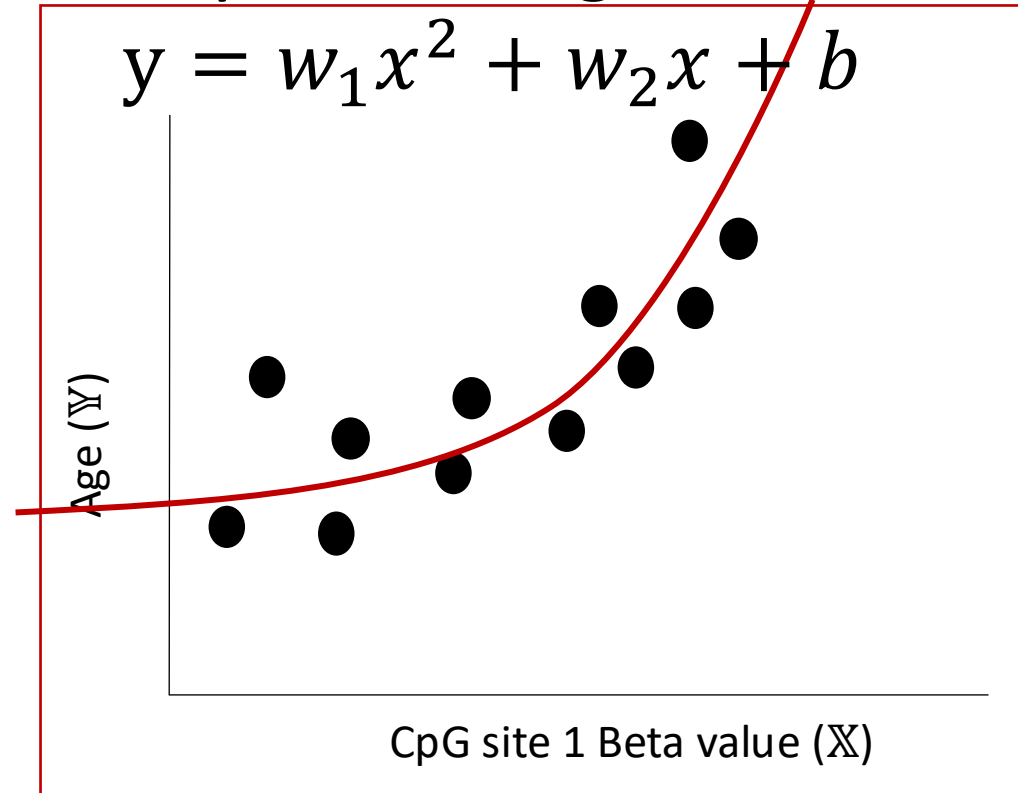
$$x^N = \dots$$

$$\mathbb{X} \in \mathbb{R}$$

Non-linear function

Polynomial regression

$$y = w_1 x^2 + w_2 x + b$$



“Age”

$$y^{(1)} = 25$$

$$y^{(2)} = 35$$

$$y^{(3)} = 40$$

⋮

⋮

⋮

⋮

$$y^N = \dots$$

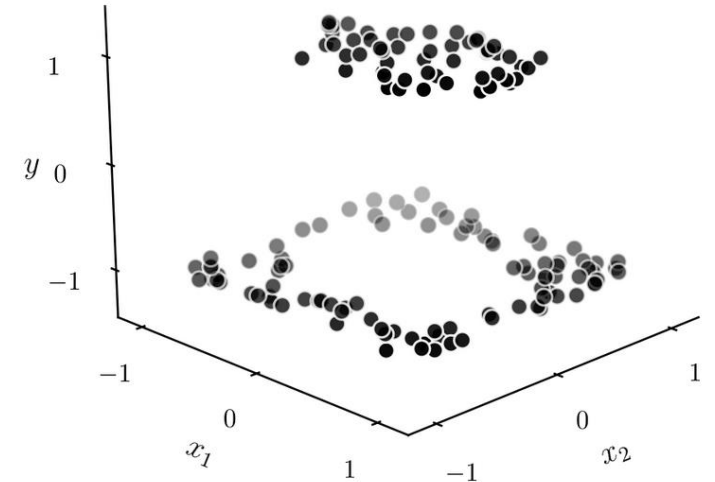
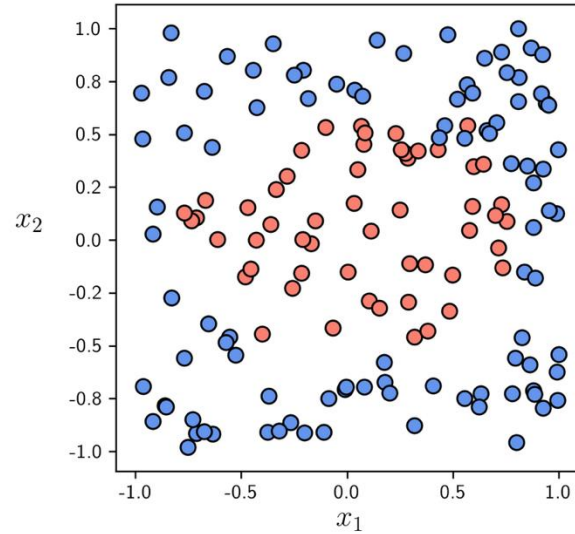
$$\mathbb{Y} \in \mathbb{R}$$

(Numerical output)

Quadratic feature expansion

Map a feature vector $\mathbf{x} = \langle x_1, x_2, x_3, \dots, x_D \rangle$ to an expanded version denoted by $\phi(\mathbf{x})$:

$$\begin{aligned}\phi(\mathbf{x}) = & \langle 1, 2x_1, 2x_2, 2x_3, \dots, 2x_D, \\ & x_1^2, x_1x_2, x_1x_3, \dots, x_1x_D, \\ & x_2x_1, x_2^2, x_2x_3, \dots, x_2x_D, \\ & x_3x_1, x_3x_2, x_3^2, \dots, x_3x_D, \\ & \dots, \\ & x_Dx_1, x_Dx_2, x_Dx_3, \dots, x_D^2 \rangle\end{aligned}$$



One can now train a linear model on this, HOWEVER...

Computational: if your learning algorithm scales linearly in the number of features, then you've just squared the amount of computation you need to perform

What issues do we run into?

Statistical: if you go by the heuristic that you should have about two examples for every feature, then you will now need quadratically many training examples in order to avoid overfitting

Kernel-based learning

Key insight: One can rewrite many linear models in a way that does not require explicitly computing $\phi(x)$.

The models discussed so far involve a product of the form $w \cdot \phi(x)$, after performing the feature mapping.

The goal is to rewrite learning algorithms so that they only ever depend on dot products between two examples, say x and z ; namely, they depend on $\phi(x) \cdot \phi(z)$

$$\begin{aligned}\phi(x) \cdot \phi(z) = & 1 + x_1 z_1 + x_2 z_2 + \cdots + x_D z_D + x_1^2 z_1^2 + \cdots + \\ & \cdots + x_D x_1 z_D z_1 + x_D x_2 z_D z_2 + \cdots + x_D^2 z_D^2\end{aligned}$$

Any questions?



Kernelized Perceptron

Original Perceptron Algorithm

Task: Binary Classification

$$y_i \in \{-1, +1\}$$

Model

$$f(x) = w \cdot x$$

Prediction

$$a = \text{sign}(w \cdot x)$$

Update rule (when model makes mistake)

$$w \leftarrow w + y_i x_i$$

Algorithm 29 PERCEPTRONTRAIN($\mathbf{D}$, $MaxIter$)

```
1:  $w \leftarrow 0, b \leftarrow 0$  // initialize weights and bias
2: for  $iter = 1 \dots MaxIter$  do
3:   for all  $(x, y) \in \mathbf{D}$  do
4:      $a \leftarrow w \cdot \phi(x) + b$  // compute activation for this example
5:     if  $ya \leq 0$  then
6:        $w \leftarrow w + y \phi(x)$  // update weights
7:        $b \leftarrow b + y$  // update bias
8:     end if
9:   end for
10: end for
11: return  $w, b$ 
```

Recap – Learn about gradient boosted classification and kernels

(1) Revisiting Gradient Boosted Regression

(2) Gradient Boosted Classification

(3) Class Activity: Gradient boosted trees

(4) Introduction to Kernels

(5) Kernelized Perceptron

Homework Reading:
[Chapter 11 in CIML Book](#)
Sections 11.1, 11.2, 11.4

Wrap up



What was the clearest point today?



menti.com
1271 3088

What was the muddiest point today?

